

Enterprise Learning & Workforce Management Platform

Milestone 1 – Landing Page & Authentication

Public Landing Page · Registration · Login · JWT Authentication · Google OAuth · Role-Based Access

Mentor: Shakti Gopal Krishnan

Team: Kartik · Kavipriya · Harshit · Gayatri

1. Objective

The objective of Milestone 1 was to design and implement the public-facing landing page and authentication system of the Enterprise Learning & Workforce Management Platform. This milestone establishes the entry point of the application. Before users can access learning features, dashboards, courses, or other protected functionality, they need a clear landing page and a secure authentication mechanism.

The main objectives were:

- Create an attractive and responsive landing page.
- Introduce users to the purpose and features of the platform.
- Provide clear navigation and calls to action.
- Implement user registration and login.
- Implement secure JWT-based authentication.
- Implement Google OAuth authentication.
- Provide role-based access for students and administrators.
- Protect private pages from unauthenticated users.
- Support both light and dark themes.
- Establish secure communication between the React frontend and Spring Boot backend.

2. Landing Page

The landing page is the first page users see when they access the platform. It acts as the public introduction to the Learning & Workforce Management Platform.

The page was developed as a React 18 single-page application using Vite. Vite was selected to provide a fast development environment and efficient hot module replacement (HMR), allowing changes made during development to appear quickly without repeatedly restarting the application.

2.1 Design and User Interface

A dark purple and magenta visual theme was implemented to give the platform a modern and professional appearance. The UI uses a custom CSS token system, which provides a centralized way of maintaining colors, spacing, typography, and other design values. This approach makes the interface more consistent because common styling values can be reused throughout different sections of the application. For example, instead of defining different colors separately on every component, common design tokens can be used across the application.

Main UI characteristics:

- Dark purple/magenta color scheme
- Modern card-based interface
- Consistent typography
- Responsive layouts
- Reusable styling
- Smooth visual hierarchy
- Modern glassmorphism-inspired elements

2.2 Decorative SVG Elements

Decorative SVG elements were added to the landing page to improve its visual appearance. These elements are mainly used as background and decorative components rather than functional content. They help create a:

- Modern appearance
- More engaging hero section
- Layered visual effect
- Glassmorphism-inspired design

The use of SVG also allows decorative graphics to remain scalable across different screen sizes.

3. Landing Page Structure

The landing page is organized into different sections to introduce the platform and guide users toward authentication.

3.1 Hero Section

The Hero section is the main introductory section of the landing page. It communicates:

- What the platform is
- What users can do on the platform
- Key course/learning highlights

- Calls to action for users

The purpose of this section is to immediately communicate the value of the platform to a new visitor. For example, the user can be guided toward actions such as:

Explore the platform → Register / Login → Start Learning

3.2 Course Highlights

The landing page also presents course-related highlights to give users an idea about the learning opportunities available on the platform. This helps users understand that the platform is not simply an authentication application but a complete learning environment.

3.3 Call-to-Action

Clear Call-to-Action (CTA) elements are provided to guide users toward the next step. For example:

- Login
- Register
- Get Started
- Explore Courses

These actions connect the public landing page with the authentication flow.

4. Responsive Design

The landing page was designed to work across different screen sizes. The layout was tested across:

- Desktop screens
- Tablet-sized screens
- Mobile screens

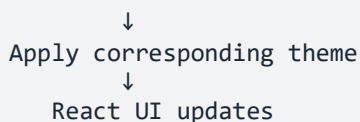
Responsive design ensures that the page remains usable even when the screen width changes. The layout adapts according to the available screen size rather than maintaining a fixed desktop-only design.

5. Theme Support

The platform supports both Dark Mode and Light Mode. A theme toggle was implemented so users can switch between the two appearances. The theme functionality is connected to the application's Auth & Theme Context, allowing the selected theme to be managed centrally.

Theme flow:

```
graph TD
    A[User clicks Theme Toggle] --> B[Theme Context]
    B --> C[Update Theme State]
```



This provides a consistent theme across the application.

6. Icons and Visual Language

Lucide React icons were used throughout the landing page. Using the same icon library provides a consistent visual language across the application. Instead of using unrelated icon styles in different sections, Lucide icons maintain a common:

- Shape
- Stroke style
- Visual weight
- Design language

This improves the overall consistency of the user interface.

7. Authentication System

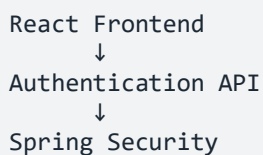
The second major component of Milestone 1 is the authentication system. Authentication is responsible for identifying users before they are allowed to access protected parts of the application. The platform supports:

- 1. Normal registration
- 2. Normal login
- 3. Google Sign-In
- 4. JWT authentication
- 5. Role-based access control
- 6. Protected routes

The authentication system is divided between the Spring Boot backend and the React frontend.

8. Backend Authentication – Spring Boot

The backend authentication system was implemented using Spring Security and JWT. The authentication architecture is:



```

↓
AuthService
↓
Validate User
↓
Generate JWT
↓
Return Token

```

The backend uses stateless JWT authentication. This means the server does not depend on maintaining a traditional server-side login session for every request. Instead, the authenticated user's JWT is used to identify and authorize subsequent requests.

9. JWT Authentication

JWT stands for JSON Web Token. After a successful login, the backend generates a JWT and sends it to the frontend. The frontend then uses the token when communicating with protected backend APIs. The overall process is:

```

User enters login details
↓
React sends login request
↓
Spring Boot receives request
↓
Credentials are validated
↓
JWT is generated
↓
JWT returned to frontend
↓
Frontend uses JWT for protected requests

```

For later requests:

```

React
↓
Authorization: Bearer <JWT>
↓
Spring Security
↓
JwtAuthFilter
↓
Validate JWT
↓
Allow / Reject Request

```

This prevents unauthorized users from directly accessing protected application functionality.

10. Core Backend Authentication Classes

Several important backend classes were implemented for authentication.

SecurityConfig.java

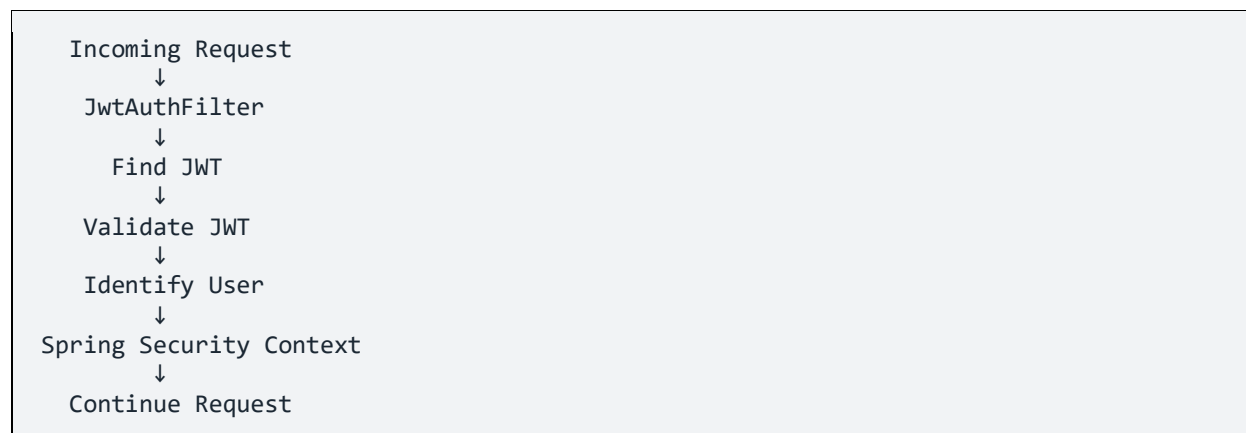
Contains the main Spring Security configuration. It is responsible for defining security rules and determining which resources require authentication. It is also involved in configuring authentication behavior, authorization rules, security filters, and CORS/security configuration.

JwtUtil.java

Responsible for JWT-related operations. Its role includes working with JWT tokens, such as generating tokens, reading token information, and validating token-related information.

JwtAuthFilter.java

Works as a security filter. When a protected API request arrives, it checks the JWT supplied by the client.

**AuthService.java**

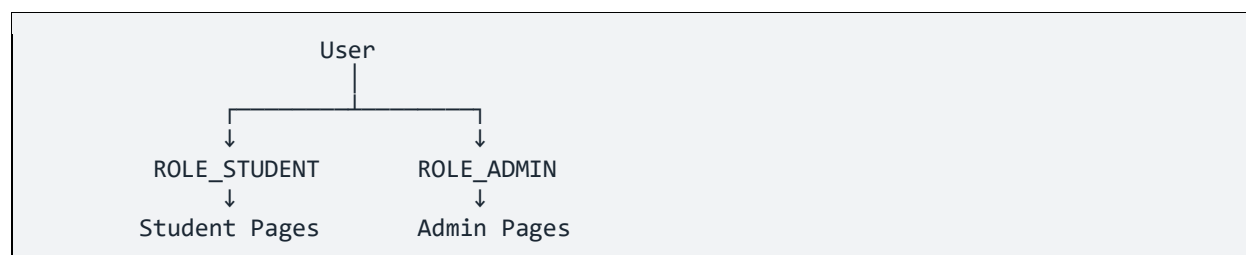
Contains the main authentication-related business logic. It handles the processing required for user authentication rather than placing all logic directly inside the controller.

AuthController.java

Exposes authentication-related REST APIs. The frontend communicates with these APIs for operations such as Login, Registration, and Authentication.

11. Role-Based Access Control

The platform implements role-based access control. Milestone 1 specifically includes `ROLE_STUDENT` and `ROLE_ADMIN`. This means the application doesn't simply determine whether a user is logged in — it can also determine what that user is allowed to access. For example:



This is important because administrative functionality should not be accessible to normal students.

12. Google OAuth Authentication

The project also supports Google Sign-In. Instead of requiring users to manually enter their platform password, users can authenticate using their Google account. The flow is:



The project uses `GoogleTokenVerifier.java` for Google authentication-related token verification. The milestone specifically includes a two-JWT flow involving access and refresh tokens for the Google OAuth integration.

13. Frontend Authentication

The React frontend contains the user-facing authentication interface. It includes:

- Login page
- Registration page
- Google Sign-In button
- Client-side validation
- Protected routing

13.1 Login Form

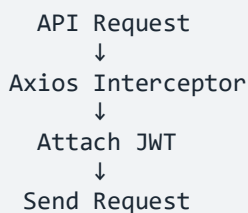
The login form allows users to enter their credentials. The frontend performs basic client-side validation before sending the request to the backend. For example, it can check whether required fields have been entered before submitting the form. Then the credentials are sent to the Spring Boot authentication API.

13.2 Registration Form

New users can create an account through the registration page. The frontend validates the registration fields and sends the information to the backend. The backend then processes the registration and creates the user account.

14. Axios Interceptors

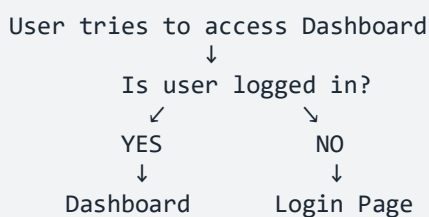
The frontend uses Axios interceptors to automatically attach the JWT bearer token to outgoing authenticated requests. Instead of manually adding the token to every API call, the interceptor can handle this automatically. Conceptually:



For example: Authorization: Bearer <JWT>. This makes communication with protected backend APIs easier and more consistent.

15. Protected Routing

The frontend also implements protected routes. A protected route means that only authenticated users should be allowed to access that page. For example:



If an unauthenticated user attempts to access a protected dashboard, the application redirects them to the appropriate authentication page instead of allowing access.

16. CORS Configuration

CORS stands for Cross-Origin Resource Sharing. Since the React frontend and Spring Boot backend run separately during development, they may use different origins/ports. For example:

```

Frontend → http://localhost:5173
Backend  → http://localhost:8080
  
```

CORS configuration allows the frontend to communicate with the backend securely. The milestone specifically included configuring CORS for communication between the React client and Spring Boot API.

17. Bug Fix – Google Sign-In Redirect

One issue identified during this milestone was that after successful Google Sign-In, the user was being redirected back to the landing page instead of the dashboard. This was fixed as part of the milestone.

Previous flow:

```
graph TD; A[Google Sign-In] --> B[Authentication Successful]; B --> C[Landing Page X];
```

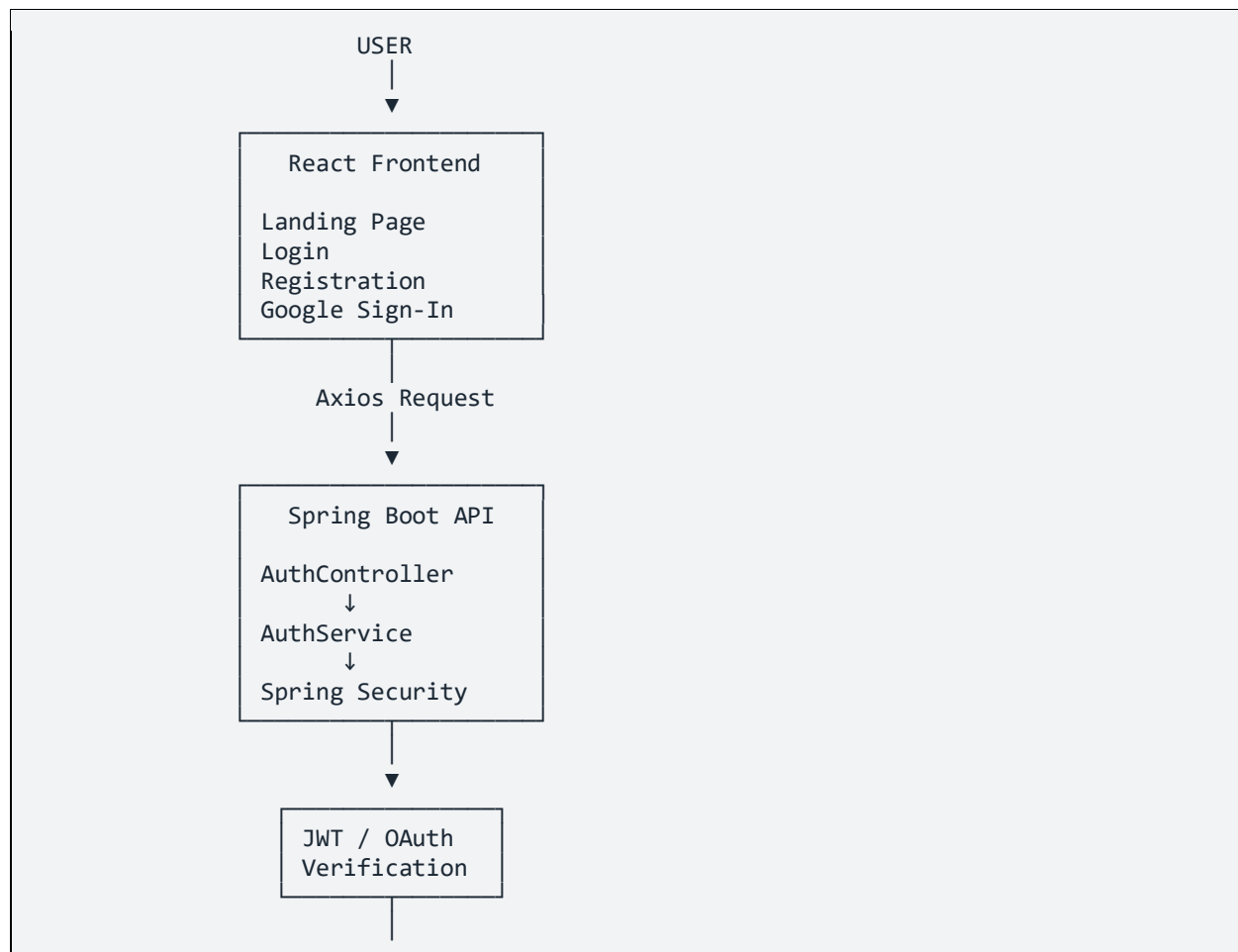
Corrected flow:

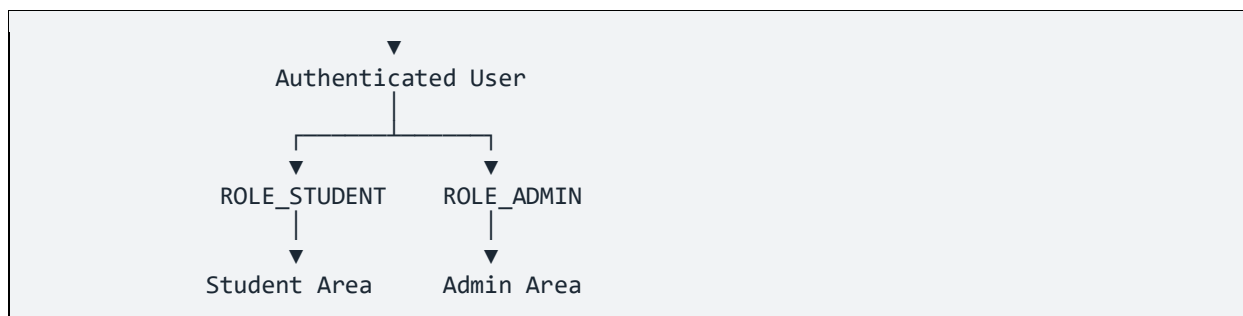
```
graph TD; A[Google Sign-In] --> B[Authentication Successful]; B --> C[JWT / User State Updated]; C --> D[Dashboard ✓];
```

This ensures that users are taken directly to the appropriate authenticated area after successful login.

18. Complete Milestone 1 Architecture

The complete authentication architecture can be represented as:





19. Conclusion

Milestone 1 successfully established the foundation of the Enterprise Learning Platform. The landing page provides the public entry point and introduces users to the platform through a responsive and modern interface. The authentication system provides secure login and registration using Spring Security and JWT, while Google OAuth provides an additional authentication option.

Role-based access control ensures that students and administrators receive the appropriate level of access. Protected routing on the frontend prevents unauthenticated users from accessing restricted pages, while Axios interceptors simplify the process of sending JWT credentials with API requests.

With the landing page and authentication system completed, the platform is now ready for the next development phase, where the core learning, course management, student dashboard, and administrative features can be built on top of this authentication foundation.